

CHAPTER 3 | Relational Query Languages

Revision.....

1. Create Table Query

Students (student_id, name, city, age)

```
CREATE TABLE students (  
    student_id INT PRIMARY KEY,  
    name VARCHAR(40),  
    city VARCHAR(30),  
    age INT  
);
```

BASIC STRUCTURE OF SQL QUERIES

The basic structure of an SQL query consists of three clauses: select, from, and where. A query takes as its input the relations listed in the from clause, operates on them as specified in the where and select clauses, and then produces a relation as the result.

E.g Displaying all Department names with instructors:

```
SELECT dept_name FROM instructors;
```

BASIC STRUCTURE OF SQL QUERIES

But, what if we wanted only the names of departments with instructors. In such case we use the **DISTINCT** keyword, which only displays unique values in the generated output set.

It's implementation is as:

```
SELECT DISTINCT dept_name
```

```
FROM instructors;
```

BASIC STRUCTURE OF SQL QUERIES

Then, what about when we explicitly want our output to show duplicates. In that case we use '**ALL**' keyword after select.

It's implementation is as:

```
SELECT ALL dept_name
```

```
FROM instructors;
```

Which gives the same output as **SELECT**

BASIC STRUCTURE OF SQL QUERIES

We can also use mathematical operations on table attributes. For instance, we can check what the salary would be if increased by 10% of all instructors as:

It's implementation is as:

```
SELECT name, salary * 1.1
```

```
FROM instructor;
```

Note that it doesn't make any changes to the SALARY attribute

BASIC STRUCTURE OF SQL QUERIES

Now, if we want to further narrow out instructors from Electrical Department, we modify our query to be as:

It's implementation is as:

```
SELECT name, salary * 1.1
```

```
FROM instructor
```

```
WHERE department = 'electrical';
```

BASIC STRUCTURE OF SQL QUERIES

Taking the initial query one step further: to retrieve all names of instructors with their name and department name and department block no, we perform

instructor (emp_id, name, dept_id, salary) | department (dept_id, name, block_no)

```
SELECT instructor.name, department.name, department.block_no
```

```
FROM instructor, department
```

```
WHERE instructor.dept_id = department.dept_id
```

BASIC STRUCTURE OF SQL QUERIES

Taking the initial query one step further: to retrieve all names of instructors with their name and department name and department block no, we perform

instructor (emp_id, name, dept_id, salary) | department (dept_id, name, block_no)

```
SELECT instructor.name, department.name, department.block_no
```

```
FROM instructor, department
```

```
WHERE instructor.dept_id = department.dept_id
```

Aggregate Functions

Aggregate functions perform calculations on multiple rows and return a single value.

It's types are:

- COUNT
- SUM
- AVG
- MIN
- MAX

Aggregate Functions

1. COUNT() - Counts rows

-- Count all rows

```
SELECT COUNT(*) FROM employees;
```

-- Count non-null values in a column

```
SELECT COUNT(email) FROM employees;
```

-- Count distinct values

```
SELECT COUNT(DISTINCT department) FROM employees;
```

Aggregate Functions

2. SUM() - Calculates total sum

-- Total salary

```
SELECT SUM(salary) FROM employees;
```

-- Sum with condition

```
SELECT SUM(salary) FROM employees WHERE  
department = 'Sales';
```

Aggregate Functions

3. AVG() - Calculates average

-- Average salary

```
SELECT AVG(salary) FROM employees;
```

-- Average with ROUND

```
SELECT ROUND(AVG(salary), 2) FROM employees;
```

Aggregate Functions

4. MIN() - Finds minimum value

-- Lowest salary

```
SELECT MIN(salary) FROM employees;
```

-- Earliest hire date

```
SELECT MIN(hire_date) FROM employees;
```

Aggregate Functions

5. MAX() - Finds maximum value

-- Highest salary

```
SELECT MAX(salary) FROM employees;
```

-- Latest hire date

```
SELECT MAX(hire_date) FROM employees;
```

The FROM keyword

-- Simple Syntax

```
SELECT * FROM employees;
```

-- Creating Aliases

```
SELECT e.first_name, e.last_name
```

```
FROM employees AS e;
```

```
Or (FROM employees e;)
```

The WHERE keyword

```
SELECT * FROM employees
```

```
WHERE department = 'Sales';
```

```
-- Using numbers
```

```
SELECT * FROM products
```

```
WHERE price > 100;
```

The WHERE keyword | Comparison equators

=	-- Equal
<> or !=	-- Not equal
>	-- Greater than
<	-- Less than
>=	-- Greater than or equal
<=	-- Less than or equal

The WHERE keyword | AND operator

```
SELECT * FROM employees
```

```
WHERE department = 'Sales'
```

```
AND salary > 50000
```

```
AND hire_date > '2023-01-01';
```

The WHERE keyword | OR operator

```
SELECT * FROM products
```

```
WHERE category = 'Electronics'
```

```
OR category = 'Computers'
```

```
OR price < 50;
```

The WHERE keyword | NOT operator

```
SELECT * FROM customers
```

```
WHERE NOT country = 'USA';
```

```
SELECT * FROM orders
```

```
WHERE NOT (status = 'Cancelled' OR  
status = 'Returned');
```

Intermediate WHERE features

1. BETWEEN

```
SELECT * FROM products
```

```
WHERE price BETWEEN 50 AND 100;
```

```
SELECT * FROM orders
```

```
WHERE order_date BETWEEN '2024-01-01' AND '2024-03-31';
```

```
SELECT * FROM employees
```

```
WHERE salary NOT BETWEEN 30000 AND 70000;
```

Intermediate WHERE features

2. IN

```
SELECT * FROM customers
```

```
WHERE country IN ('USA', 'Canada', 'UK');
```

```
SELECT * FROM products
```

```
WHERE category_id IN (1, 3, 5, 7);
```

Intermediate WHERE features

2. IN

```
SELECT * FROM employees
WHERE department_id IN (
    SELECT department_id FROM departments
    WHERE location = 'New York');
```

Intermediate WHERE features | LIKE

Wildcard	Meaning	Example	Matches	Doesn't Match
%	Zero or more characters	Jo%'	John, Joe, Jonathan	Alex, Bob
_	Exactly one character	J_n'	Jen, Jon, Jin	John, Joan
[abc]	One character from set	[B]ill'	Bill, Jill	Mill, Will
[a-c]	One character in range	[A-C]at'	Aat, Bat, Cat	Dat, Eat
[^abc]	Not in set	[^BM]ill'	Mill, Will	Bill, Mill

Question

Employee (emp_id, first_name, last_name, email, age, department, salary)

Write an SQL Query to

1. Display all employees with salary less than 20,000 and age >25.
2. Display all names having ACEM email.
3. Display all names having email in Gmail or Hotmail
4. Display all employees whose first name starts with the letter 'L'.
5. Display all the employees who are from the departments:
Electrical, Electronics or Communication (use IN keyword)

Basic Structure of SQL Queries

-- Find products more expensive than the most expensive product in category 1

```
SELECT * FROM products
```

```
WHERE price > (
```

```
    SELECT MAX(price) FROM products WHERE  
    category_id = 1
```

```
);
```

Basic Structure of SQL Queries

```
-- Find employees earning more than average  
  
SELECT * FROM employees  
  
WHERE salary > (  
    SELECT AVG(salary) FROM employees  
  
);
```

The GROUP BY keyword

It groups rows that have the same values in specified columns into summary rows.

```
SELECT column1, aggregate_function(column2)  
  
FROM table  
  
GROUP BY column1;
```

Write the Query of the following output

```
overall_average_marks
```

```
86.64
```

Write the query that displays the above output.

```
SELECT AVG(marks) as overall_average_marks  
FROM student;
```

Using GROUP BY

name	department	marks
Alice	Computer Science	80
Bob	Mathematics	90
Carol	Computer Science	90
David	Physics	80
Emma	Mathematics	90
Frank	Physics	90
Grace	Computer Science	100

Then, upon using the Group By keyword, we get the output as:

```
SELECT department, AVG(marks) as dept_avg, COUNT(*) as  
count  
FROM student  
GROUP BY department;
```

department	dept_avg	count
Computer Science	90	3
Mathematics	90	2
Physics	85	2

The HAVING keyword

HAVING is a SQL clause used to filter groups of rows after aggregation has been performed by the GROUP BY clause. It operates on the aggregated results, not on individual rows.

```
SELECT column1, aggregate_function(column2)  
FROM table  
GROUP BY column1  
HAVING condition;
```

Question

1. Show all students with marks greater than 85.
2. Display students from 'Mumbai' or 'Delhi'.
3. Find students whose names start with 'A'.
4. Show students in class '11' who have marks less than 80.
5. Find the total number of students in each house.
6. Find the average marks of students in each class
7. Count the number of students from each city.
8. Display the maximum marks scored by students in each house.
9. Show houses where the average marks of students are above 85.
10. Find classes having more than 3 students.

NESTED SUB QUERIES

...

JOINS in SQL

A database operation that combines rows from two or more tables based on a related column between them, so you can see connected data together.

There are 4 types of joins:

1. Inner Join
2. Left Join
3. Right Join
4. Full (Outer) Join

Database Modification

Database modification means changing the data or structure of a database using commands. It involves adding, updating, or removing data (and sometimes tables).

1. INSERT — Add New Data
2. UPDATE — Change Existing Data
3. DELETE — Remove Data

Insert

```
INSERT INTO Customers (Name,  
Email)
```

```
VALUES ( 'Anna' , ' anna@email.com' );
```

Update

```
UPDATE Customers
```

```
SET Email = 'john.new@email.com'
```

```
WHERE Name = 'John';
```

Always use WHERE with UPDATE/DELETE unless you want to change every row.

Delete

```
DELETE FROM Customers  
WHERE Name = 'Anna';
```

VIEWS

A view is a virtual table based on the resultset of a SQL query. It doesn't store data physically (except materialized views) but acts as a saved query that you can reference like a table.

So,

It's a virtual table whose contents are dynamically derived from base tables when referenced.

VIEWS

Syntax:

```
CREATE VIEW view_name AS
```

```
SELECT column1, column2, ...
```

```
FROM table_name
```

```
WHERE condition;
```

VIEWS | Significance

1. **Security**

It only lets certain group of people certain part of your database.

2. **Data Abstraction**

It hides the technical details so users see clean, simple data.

3. **Hides complexity**

It turns complex database operations into simple tables.

VIEW | Materialised View

A materialized view saves query results as an actual table in your database. It's a pre-calculated snapshot of data that you can query instantly.

Unlike regular views, it stores data physically in a table and gives the same set of information and may require frequent updating.

VIEW | Materialised View

It's characteristics is:

- Stores Results Physically
- Much Faster Queries
- Needs Manual Updates

Privileges and Role Management

Permissions that control what users can do in a database. They are:

- SELECT – Read data
- INSERT – Add new data
- UPDATE – Modify existing data
- DELETE – Remove data
- CREATE – Make new tables/databases
- DROP – Delete tables/databases
- EXECUTE – Run stored procedures

Role Management

Groups of privileges that you can assign to users.

Instead of giving permissions to each user individually, you:

1. Create a role (like "Manager" or "Developer")

```
CREATE ROLE manager;
```

2. Give permissions to the role

```
GRANT SELECT, INSERT, UPDATE ON students  
TO manager;
```

3. Assign users to the role

```
GRANT manager TO 'user1'@'localhost';
```

Stored Procedures

A stored procedure is a precompiled set of SQL statements that is stored in the database and can be executed by calling its name.

It can accept input parameters, perform operations (queries, inserts, updates, deletes), and return results or output parameters.

Stored Procedures Example

```
DELIMITER $$
```

```
CREATE PROCEDURE get_students_by_city(IN city_name VARCHAR(30))
```

```
BEGIN
```

```
    SELECT name, class, marks
```

```
    FROM students
```

```
    WHERE city = city_name;
```

```
END $$
```

```
DELIMITER ;
```

```
CALL get_students_by_city('Delhi');
```

Significance of Stored Procedures

- **Performance:** Stored procedures are precompiled, so execution is faster than running individual SQL statements repeatedly.
- **Reusability:** Write once, use many times from different applications or queries.
- **Security:** Users can be granted permission to execute a procedure without giving direct access to tables.

Performance of Stored Procedures

For each query in SQL,

1. Syntax check
2. Parse query
3. Optimize execution plan
4. Compile
5. Execute

Vs Stored Procedures

1. Compile once
2. Store execution Plan in cache
3. Execute plan directly

Triggers

A trigger is a database object that automatically executes when a specified event (INSERT, UPDATE, DELETE) occurs on a table.

They are not assigned to users instead, you attach them to tables so the same logic applies to everyone.

Trigger example

```
CREATE TABLE students (  
    roll_no INT PRIMARY KEY,  
    name VARCHAR(40),  
    marks INT  
);
```

Trigger example

```
DELIMITER $$  
  
CREATE TRIGGER  
before_insert_students  
  
BEFORE INSERT  
  
ON students  
  
FOR EACH ROW  
  
BEGIN
```

```
    IF NEW.marks < 0  
    THEN  
        SET NEW.marks =  
0;  
    END IF;  
END $$  
  
DELIMITER ;
```

Trigger use case scenario

```
INSERT INTO students VALUES (1, 'Ram', 85);
```

```
INSERT INTO students VALUES (2, 'Hari', -10);
```

Output:

roll_no	name	marks
1	Ram	85
2	Hari	0

What is Relational Algebra?

It is a formal, procedural query language that provides the mathematical foundation for RDBMS.

It is a collection of operations that take one or two relations (tables) as input and produce a new relation as output.

Unlike SQL, which is Declarative (you describe what you want), Relational Algebra is Procedural. It defines a specific sequence of algebraic steps to arrive at a result.

Selection Operator

The Selection Operator is one of the most fundamental unary operators in relational algebra. It is used to retrieve specific rows (tuples) from a relation that satisfy a given condition.

Symbol: Represented by the lowercase Greek letter Sigma (σ)

SQL Equivalent: It maps directly to the `WHERE` clause in an SQL query.

Projection Operator

The Projection Operator is the "vertical" counterpart to the Selection operator. Projection filters columns, allowing us to focus only on the specific attributes you need.

Symbol: Represented by the lowercase Greek letter Pi (π)

SQL Equivalent: It maps directly to the SELECT clause in an SQL query.

Question #1

Student | sid | sname | major

Select all student data of Alice.

SELECT *

FROM Student

WHERE sname = 'Alice';

In relational algebra:

$\sigma_{\text{sname} = \text{'Alice'}}(\text{Student})$

Question #2

SportsTeam | team_id | team_name | sid

List only the names of all sports teams.

```
SELECT team_name
```

```
FROM SportsTeam
```

In relational algebra:

$$\pi_{\text{team_name}}(\text{SportsTeam})$$

Question #3

SELECT sname

FROM Student

WHERE major = 'CS';

In relational algebra:

$\pi_{\text{sname}} (\sigma_{\text{major} = \text{'CS'}}(\text{Student}))$

Question #4

```
SELECT *  
  
FROM SportsTeam  
  
WHERE team_name = 'Soccer' OR sid = 1;
```

In relational algebra:

$$\sigma_{\text{team_name} = \text{'Soccer'} \vee \text{sid} = 1}(\text{SportsTeam})$$

Question #5

```
SELECT *  
  
FROM SportsTeam  
  
WHERE team_name = 'Soccer' AND sid = 1;
```

In relational algebra:

$$\sigma_{\text{team_name} = \text{'Soccer'} \wedge \text{sid} = 1}(\text{SportsTeam})$$

Question #6

Write a query that finds students whose major is NOT 'Math'

Select *

From students

Where major not in ('Math')

In relational algebra:

Students - $\sigma_{\text{major} = \text{'Maths'}}(\text{Students})$

or $\sigma_{\neg(\text{major} = \text{'Maths'}})}(\text{Students})$

Join Operator

The Join Operator is a binary operator used to combine related tuples from two different relations into single tuples. It is essentially a cross product followed by a selection condition.

Symbol: Represented by the "bowtie" symbol $\bowtie$.

SQL Equivalent: It maps to the JOIN clause (specifically INNER JOIN for the natural join).

Question #7

Show a complete list of all students and the teams they are on.

```
SELECT * FROM Student  
JOIN SportsTeam;
```

In relational algebra:

Student $\bowtie$ SportsTeam

Question #8

Find the student names and team names for everyone in the 'CS' major.

```
SELECT sname, team_name
```

```
FROM Student
```

```
JOIN SportsTeam ON Student.sid = SportsTeam.sid
```

```
WHERE major = 'CS';
```

In relational algebra:

$$\pi_{\text{sname, team_name}}(\sigma_{\text{major='CS'}}(\text{Student} \bowtie \text{SportsTeam}))$$

Question #9

List the team_id and team_name for all teams that have students with an ID > 5.

```
SELECT team_id, team_name
FROM SportsTeam
JOIN Student ON SportsTeam.sid = Student.sid
WHERE Student.sid > 5;
```

In relational algebra:

$$\pi_{\text{team_id, team_name}}(\sigma_{\text{sid} > 5}(\text{SportsTeam} \bowtie_{\text{SportsTeam.sid} = \text{Student.sid}} \text{Student}))$$

Rename Operator

The Rename operator is a **unary operator** (meaning it operates on a single relation). It is used to change the name of a relation or the names of the attributes within that relation. This is particularly useful for self-joins or for making the results of complex operations more readable.

Symbol: Represented by the Greek letter rho: ρ

SQL Equivalent: It maps to the **AS** keyword used for aliasing tables or columns.

Syntax: $*_{\rho_S}(R)$: Renames relation R to S

Rename Operator

- $\rho_{S(A_1, A_2)}(R)$

Renames the relation to S and its attributes to A_1, A_2

Example: If you have a table Emp(ID, Name) and you want to rename it to Staff with columns Staff_ID and Staff_Name:

```
SELECT ID AS Staff_ID, Name AS Staff_Name  
FROM Emp AS Staff;
```

Answer: $\rho_{\text{Staff}(\text{Staff_ID}, \text{Staff_Name})}(\text{Emp})$

Assignment Operator

It is used to break down complex relational algebra expressions into smaller, manageable parts. It allows you to store the result of an expression in a temporary relation variable, which can then be used in subsequent operations.

- **Symbol:** Represented by the left-pointing arrow [$\leftarrow$].
- **Purpose:** To improve readability and modularity. Instead of writing one massive, nested expression, you can assign intermediate steps to temporary names.
- **Syntax:** Temp_Relation $\leftarrow$ Relational Algebra Expression

Assignment Operator

(A) WITH CS_Students AS (
 SELECT * FROM Students WHERE Dept = 'CS'
)

SELECT Name from CS_Students;

(B) SELECT Name FROM (
 SELECT * FROM Students WHERE Dept = 'CS')
AS CS_Students;

$CS_Students \leftarrow \sigma_{Dept = 'CS'}(Students)$
 $\pi_{Name}(CS_Students)$

Questions

- Employees [EmpID | Name | Salary | DeptID | ManagerID]
 - Departments [DeptID | DeptName | Location]
 - Projects [ProjID | ProjName | DeptID]
-
1. **Selection:** Find all details of employees who earn more than \$70,000.
 2. **Projection:** List only the **Name** and **Salary** of all employees.
 3. **Rename:** Rename the **Location** attribute in the **Departments** table to **City**.

Students (student_id, name, city, age)

Courses (course_id, course_name, fee)

Enrollments (enroll_id, student_id, course_id, enroll_date)

Q1. Display all columns of all students.

Q2. Display the names and cities of students who live in Kathmandu.

Q3. Display students whose age is greater than 20.

*Q4. Display student names and ages, sorted by age in descending order.

Q5. Display all courses whose fee is greater than 30,000.

*Q6. Display the top 2 most expensive courses.

Q7. Display student names along with the courses they are enrolled in.

Q8. Display names of students who are enrolled in the course named 'Database Systems'.

*Q9. Display the latest 3 enrollments showing student name, course name, and enrollment date.

*Q10. Display each course name and the total number of students enrolled in it.

Gamma Operator

The γ (gamma) operator is used to group tuples based on one or more attributes and apply aggregate functions (such as COUNT, SUM, AVG, MIN, MAX) on each group.

- **Symbol:** Represented by the gamma symbol (γ).
- **Purpose:** To perform aggregation & to implement GROUP BY and aggregate functions
- **Example:** *Count the number of students enrolled in each course*

$\gamma_{\text{course_id}, \text{COUNT}(\text{student_id}) \rightarrow \text{total_students}}(\text{Enrollments})$

Employees (emp_id, name, department, salary)
Projects (project_id, project_name, budget)
Assignments (assign_id, emp_id, project_id, role)

- Q1. Display all distinct departments from the employees table.
- Q2. Display names and salaries of employees whose salary is between 40,000 and 70,000.
- Q3. Display employees whose name starts with the letter 'A'.
- Q4. Display employees who work in either 'HR' or 'Finance' departments.
- Q5. Display the top 3 highest-paid employees.
- Q6. Display employee names along with the projects they are assigned to.
- Q7. Display all employees, including those who are not assigned to any project.
- Q8. Display names of employees working on the project named 'ERP System'.
- Q9. Display each department and the number of distinct employees in it.
- Q10. Display project names and the total number of employees assigned to each project.